

SDC Assignment 4 - Implement Kalman Filter

Ying Pei Lin

Spring 2026

1 Code Explanation

The Kalman Filter is implemented based on the algorithm provided on the HackMD site, as detailed below.

Algorithm 1 Kalman Filter

```
1: Algorithm Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:    $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$   
3:    $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$   
4:    $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$   
5:    $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$   
6:    $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$   
7:   return  $\mu_t, \Sigma_t$ 
```

```
class KalmanFilter:  
    def __init__(self, x=0, y=0, yaw=0):  
        self.state = np.array([x, y, yaw])  
        self.A = np.identity(3)  
        self.B = np.identity(3)  
        self.S = np.identity(3) * 1  
        self.C = np.array([[1.0, 0.0, 0.0], [0.0, 1.0, 0.0]])  
        self.R = np.identity(3) * 1  
        self.Q = np.identity(2) * 1  
  
    def predict(self, u):  
        self.state = self.A @ self.state + self.B @ u  
        self.S = self.A @ self.S @ self.A.T + self.R  
        return self.state, self.S  
  
    def update(self, z):  
        K = self.S @ self.C.T @ np.linalg.inv(self.C @ self.S @ self.C.T + self.Q)  
        y = z - self.C @ self.state  
        self.state = self.state + K @ y  
        self.S = (np.identity(3) - K @ self.C) @ self.S  
        return self.state, self.S
```

Listing 1: Python implementation of Kalman Filter

The prediction step corresponds to Lines 2-3 in Algorithm 1 and the `predict(self, u)` method in Listing 1. It estimates the current state using the previous state and the control input before incorporating the new measurement. The key variables in the prediction step are:

- $\bar{\mu}_t$: The predicted state, with shape 3×1 . It propagates the previous state μ_{t-1} using the transition matrix A_t and adds the control input u_t scaled by the control matrix B_t .
- $\bar{\Sigma}_t$: The predicted covariance, with shape 3×3 . It projects the previous uncertainty Σ_{t-1} forward through the system dynamics using $A_t \Sigma_{t-1} A_t^T$ (refer to Appendix A for detailed derivation) and adds the process noise R_t .

The update step corresponds to Lines 4-6 in Algorithm 1 and the `update(self, z)` method in Listing 1. It corrects the prediction using the actual measurement z_t . The key variables in the update step are:

- K_t : The Kalman gain, with shape 3×2 . It computes the optimal weighting that minimizes the *a posteriori* error covariance by balancing the predicted state uncertainty $\bar{\Sigma}_t$ against the measurement noise Q_t (refer to the Appendix B for detailed derivation).
- μ_t : The updated state, with shape 3×1 . It corrects the predicted state $\bar{\mu}_t$ by adding the measurement residual $(z_t - C_t \bar{\mu}_t)$ proportionally scaled by K_t .
- Σ_t : The updated covariance, with shape 3×3 . It computes the reduced *a posteriori* state uncertainty after incorporating the new measurement.

2 Filtered Path Result

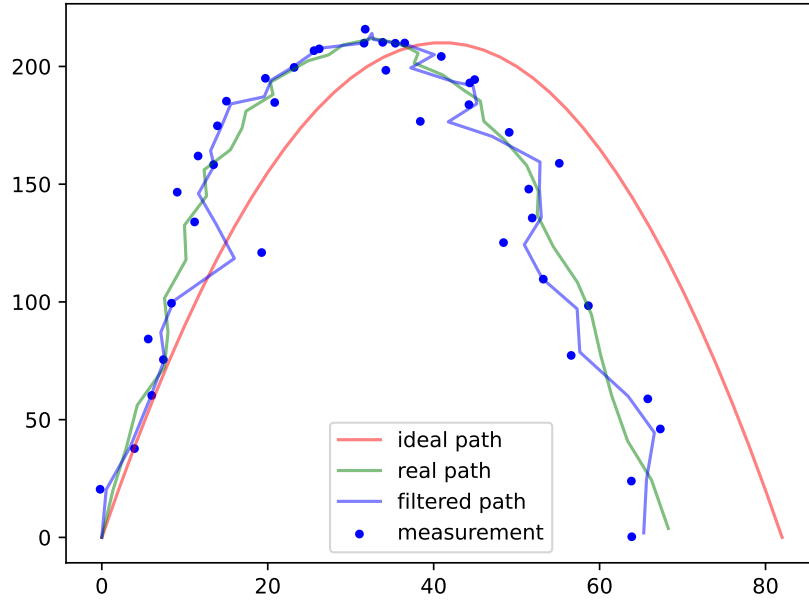


Figure 1: The filtered path result compared to the ideal path, real path, and noisy measurements.

3 Design of Observation Matrix (C)

The observation matrix C maps the system state vector $\mu = [x, y, \text{yaw}]^T$ into the observed measurement space. Because the sensor only measures positional coordinates (x, y) and cannot directly measure the heading (yaw), C is designed as a 2×3 matrix to extract only the observable variables.

$$C = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \end{bmatrix}$$

This formulation ensures that the predicted measurement $C\bar{\mu}$ correctly filters out the unobservable yaw, outputting only the x and y components required for the residual calculation:

$$C\bar{\mu} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 \end{bmatrix} \begin{bmatrix} \bar{x} \\ \bar{y} \\ \bar{\text{yaw}} \end{bmatrix} = \begin{bmatrix} \bar{x} \\ \bar{y} \end{bmatrix}$$

4 Design of Covariance Matrices (Q, R)

The covariance matrices, R and Q , represent the uncertainty in the system's process and measurements, respectively. The process noise covariance R accounts for unmodeled dynamics in the state transition and is initialized as a 3×3 identity matrix. This baseline assumes that the prediction errors for x , y , and yaw are independent and have an equal variance of 1:

$$R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Similarly, the measurement noise covariance Q quantifies the unreliability of the sensor data. Because the measurement vector only contains the (x, y) coordinates, Q is initialized as a 2×2 identity matrix. This assumes the sensor errors for the x and y axes are independent and equally distributed with a variance of 1:

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

5 Analysis: Q=0.001, R=1000

Setting $Q = 0.001$ and $R = 1000$ means the filter highly trusts the sensor measurements over the dynamic model. The Kalman gain increases, forcing the state to follow the noisy data. This results in a jagged, oscillatory path that fails to smooth out measurement errors.

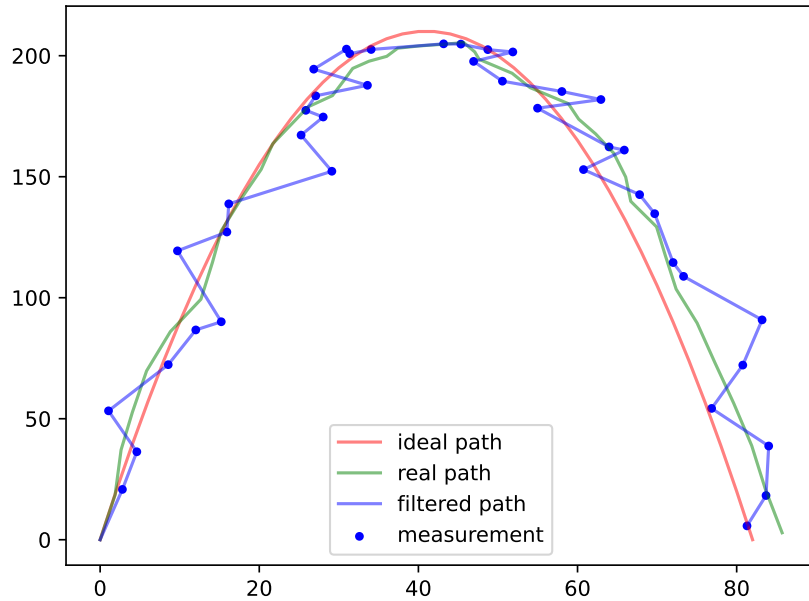


Figure 2: The filtered path result with low measurement noise and high process noise.

6 Analysis: Q=1000, R=0.001

Setting $Q = 1000$ and $R = 0.001$ indicates the filter relies heavily on its dynamic model and largely ignores sensor measurements. The Kalman gain decreases significantly. This produces an extremely smooth path, but it suffers from latency and deviates from the true path if the model is inaccurate.

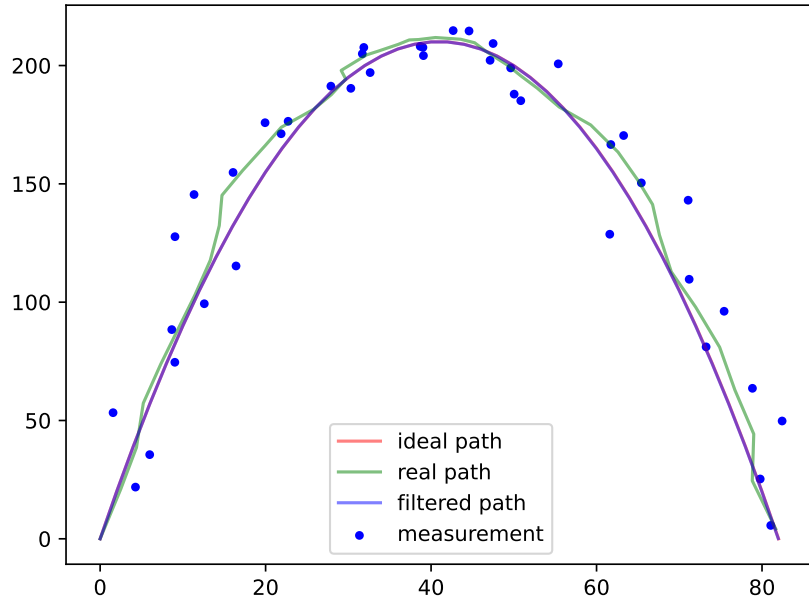


Figure 3: The filtered path result with high measurement noise and low process noise.

7 Impact of Q and R on Filter Output

The filter's performance is fundamentally determined by the ratio of Q to R , which controls the Kalman gain. Tuning these matrices involves a critical trade-off: emphasizing the model (low R , high Q) yields a smoother but lagging trajectory, while emphasizing the sensor (high R , low Q) creates a responsive but noisy path.

A Derivation of Covariance Projection

The prediction of the covariance matrix involves the term $A_t \Sigma_{t-1} A_t^T$. This is based on the property of linear transformations on random variables.

If a random vector x has a covariance matrix Σ_x , and undergoes a linear transformation $y = Ax$, the covariance of the new vector y is calculated as follows:

$$\begin{aligned}\Sigma_y &= E[(y - \mu_y)(y - \mu_y)^T] \\ &= E[(Ax - A\mu_x)(Ax - A\mu_x)^T] \\ &= E[A(x - \mu_x)(A(x - \mu_x))^T] \\ &= E[A(x - \mu_x)(x - \mu_x)^T A^T]\end{aligned}$$

Since A is a constant matrix in this context, it can be factored out of the expectation operator E :

$$\begin{aligned}\Sigma_y &= A \cdot E[(x - \mu_x)(x - \mu_x)^T] \cdot A^T \\ &= A \Sigma_x A^T\end{aligned}$$

In the context of the Kalman Filter, A_t governs how the system evolves over time. The operation $A_t \Sigma_{t-1} A_t^T$ effectively translates, scales, and rotates the uncertainty region (covariance ellipsoid) from time $t - 1$ into the expected state space at time t .

B Derivation of the Kalman Gain

The goal of the Kalman filter is to minimize the *a posteriori* error covariance. Let S be the *a priori* (predicted) state covariance, C be the observation matrix, and Q be the measurement noise covariance.

The *a posteriori* (updated) state estimate is defined as:

$$\mu^+ = \mu + K(z - C\mu)$$

The *a posteriori* error covariance matrix S^+ can be expanded using the definition of covariance and the state update equation. The general form is:

$$S^+ = (I - KC)S(I - KC)^T + KQK^T$$

Expanding this equation yields:

$$S^+ = S - KCS - SC^T K^T + K(CSC^T + Q)K^T$$

To find the optimal Kalman gain K that minimizes the estimation error, we minimize the trace of S^+ (since the trace represents the sum of the variances of the state variables). Let $J = \text{tr}(S^+)$ be the cost function:

$$J = \text{tr}(S) - \text{tr}(KCS) - \text{tr}(SC^T K^T) + \text{tr}(K(CSC^T + Q)K^T)$$

Using matrix trace derivative properties ($\frac{\partial \text{tr}(AB)}{\partial A} = B^T$ and $\frac{\partial \text{tr}(ABA^T)}{\partial A} = 2AB$ for symmetric B), we take the derivative of J with respect to K :

$$\frac{\partial J}{\partial K} = -2(CS)^T + 2K(CSC^T + Q)$$

To find the minimum, we set the derivative to zero:

$$-2SC^T + 2K(CSC^T + Q) = 0$$

$$K(CSC^T + Q) = SC^T$$

Right-multiplying both sides by the inverse of the innovation covariance $(CSC^T + Q)^{-1}$ gives the optimal Kalman gain:

$$K = SC^T(CSC^T + Q)^{-1}$$